

Kafka: Novice to Platform Engineer/System Architect (7 YOE)

This handbook is designed to take you from first principles to production architecture decisions expected from a platform engineer or system architect.

It is based on consolidated guidance from Apache Kafka documentation and mature production practices commonly taught in advanced Kafka architecture/security/operations tracks.

0) How to Use This Guide

Read in levels:

- Level 1 (Beginner): sections 1 to 5
- Level 2 (Intermediate): sections 6 to 11
- Level 3 (Senior/Architect): sections 12 to 23

For interview prep:

- Study the "Pitfalls" and "Trade-offs" bullets in every section.
 - Practice explaining "why this setting" instead of only naming settings.
-

1) Kafka in One Minute

Kafka is a distributed event streaming platform that stores records durably in partitioned logs and lets many producers/consumers process those records independently.

Core objects:

- Record: key, value, headers, timestamp
- Topic: logical stream name
- Partition: ordered, append-only shard of a topic
- Offset: monotonic position of record within partition
- Broker: Kafka server node
- Consumer group: shared subscription for parallel processing

Key truth:

- Ordering is guaranteed only within one partition.
-

2) Why Kafka vs Traditional MQ

Kafka excels when you need:

- High-throughput event ingestion
- Replay/reprocessing from stored history
- Fan-out to many consumers
- Event backbone for microservices/data platform

Traditional queue strengths:

- Simple work-queue semantics

- Often easier per-message ack/routing patterns

Architect decision heuristic:

- If replayable event log + independent consumers matters, Kafka is a strong default.
-

3) Core Architecture

3.1 Data Plane

- Producers append to topic partitions.
- Each partition has leader + follower replicas.
- Consumers fetch from leaders.

3.2 Replication

- Followers replicate leader log.
- ISR (in-sync replicas) are followers close enough to leader.
- High watermark (HW) marks committed visibility boundary.

3.3 Control Plane (KRaft)

- Modern Kafka uses KRaft (no ZooKeeper).
- Controller quorum stores cluster metadata in internal metadata log.
- Active controller manages metadata changes and leadership decisions.

Why it matters:

- Metadata quorum health is as important as data brokers.
-

4) Topic, Partition, Offset Deep Dive

Topics and Partitions

- Topic is logical namespace.
- Partition is unit of parallelism and ordering.

Offsets

- Offsets are per-partition, never reused.
- Consumers track offsets to resume/replay.

Log Segments

- Partitions are split into segment files.
- Retention/compaction works at segment lifecycle level.

Pitfall:

- Too many partitions can overload metadata/control plane and increase operational cost.
-

5) Producers: Correctness and Throughput

5.1 Essential Settings

- `acks=all`
- `enable.idempotence=true`

- `retries` > 0
- sensible `delivery.timeout.ms`

5.2 Throughput Tuning

- `batch.size` : bigger batches, higher compression efficiency
- `linger.ms` : wait briefly to fill batches
- `compression.type` : usually `snappy` , `lz4` , or `zstd`

5.3 Partitioning Strategy

- Keyed partitioning preserves per-key order.
- Random/no-key balances load but no entity-level ordering.

Pitfalls:

- Hot keys create hot partitions.
 - Increasing `linger.ms` blindly can break low-latency SLOs.
-

6) Consumers and Group Protocol

6.1 Group Mechanics

- One partition can be consumed by at most one consumer in a group.
- Max parallelism in a group is partition count.

6.2 Rebalances

- Triggered by member changes, topic metadata changes, or failures.
- Can pause processing if not tuned.

6.3 Assignment Strategies

- Range
- Round-robin
- Sticky
- Cooperative-sticky (often best for minimizing disruptive revokes)

6.4 Offset Commit Strategy

- Auto commit: easy, less control
- Manual commit after successful processing: safer at-least-once pattern

Pitfall:

- Committing before processing gives at-most-once and risks message loss.
-

7) Delivery Semantics and EOS Boundaries

At-most-once

- Commit/read progression before processing.
- Lowest duplication, possible loss.

At-least-once

- Process then commit.
- No silent loss in normal paths, duplicates possible.

Exactly-once in Kafka

- Requires idempotent producer + transactions + proper consumer isolation.
- Strong for Kafka-to-Kafka pipelines.

Critical boundary:

- Kafka EOS does not make external side effects (email, payment API, DB without idempotency) magically exactly-once.

Architect pattern:

- Build business idempotency keys regardless of transport semantics.
-

8) Retention, Compaction, and Data Lifecycle

Retention policies:

- Time-based (`retention.ms`)
- Size-based (`retention.bytes`)

Compaction:

- Keeps latest value per key.
- Tombstones model deletes.

Use cases:

- Event history/replay -> retention topics
- State snapshot/materialized view changelog -> compacted topics

Pitfall:

- Compaction is not immediate; stale keys may remain until cleaner runs.
-

9) Schema Governance (Avro/Protobuf/JSON Schema)

Why schema governance matters:

- Prevents consumer breakage
- Enables safe independent deployments

Compatibility modes (registry dependent):

- Backward
- Forward
- Full
- Transitive variants

Rules that scale:

- Never remove/rename fields casually.
- Prefer additive evolution with defaults.
- Enforce compatibility checks in CI/CD.

Architect anti-pattern:

- "Schema as code optional" leads to hidden coupling and outages.

10) Kafka Connect, Streams, and ksqlDB

Kafka Connect

- Source connectors ingest external data into Kafka.
- Sink connectors deliver Kafka data to external systems.

Use when:

- You want managed data movement without writing custom ingestion apps.

Kafka Streams

- Java library for stream processing (joins, windows, aggregations).

ksqlDB

- SQL interface for stream processing.

Architect lens:

- Choose the highest abstraction that still gives required control and operability.
-

11) Security End to End

Security dimensions:

- Authentication (who are you)
- Authorization (what can you do)
- Encryption in transit (TLS)
- Encryption at rest
- Auditability

Common controls:

- SASL/SCRAM or mTLS for clients
- ACL or RBAC model
- Separate principals per app/team
- Secret rotation and short-lived credentials

Trade-off:

- Strong crypto/auth can add CPU overhead; plan capacity accordingly.
-

12) Capacity Planning (Senior Level)

Plan for:

- Throughput (MB/s in and out)
- Partition count growth
- Retention storage
- Replication factor cost
- Peak vs steady traffic

Back-of-envelope storage estimate:

```
storage ~= ingress_bytes_per_day * retention_days * replication_factor * overhead_factor
```

Where overhead includes indexes, compression variance, safety margin.

Key practical rules:

- Prefer SSD/NVMe for demanding workloads.
- Avoid tiny brokers with massive partition counts.
- Leave headroom for rebalance and broker failures.

13) Broker and Cluster Configuration Strategy

Common production defaults (context-dependent):

- Replication factor: 3
- `min.insync.replicas : 2`
- `Producer acks=all`
- Controlled/unplanned shutdown tuned for safety

Placement concerns:

- Rack awareness / AZ awareness
- Even partition replica distribution

When failures happen:

- Understand leader election behavior.
- Keep durability settings aligned with business RPO.

Pitfall:

- High availability settings that silently permit data loss under minority failures.

14) KRaft Internals You Must Know

KRaft concepts:

- Controller quorum voters
- Active controller leadership
- Metadata log + snapshots
- Quorum-based metadata commit

Operational implications:

- Keep odd number of controllers (for quorum majority).
- Monitor controller quorum health separately from data-plane metrics.
- Snapshot/metadata lag issues can affect cluster stability.

Interview-ready explanation:

- "KRaft brings metadata consensus into Kafka itself, reducing external dependencies and improving scalability/operability, but controller quorum becomes a critical reliability tier."

15) Reliability Engineering and Failure Modes

Failure classes:

- Broker failure

- Network partition
- Controller quorum instability
- Disk pressure and I/O saturation
- Consumer lag runaway
- Rebalance storms

Controls:

- Retry with jitter and max attempts
- DLQ or quarantine topics
- Backpressure controls (pause/resume, bounded workers)
- Alerting on ISR shrink, under-replicated partitions, disk watermark, lag slope

Architect mindset:

- Design for "degrade gracefully" not only "works in happy path".
-

16) Multi-Cluster, DR, and Geo Strategy

Patterns:

- Active-passive DR
- Active-active regional architecture
- Data localization per jurisdiction
- Cluster linking / replication tools

Design dimensions:

- RPO/RTO targets
- Ordering guarantees across regions
- Duplicate handling during failover/failback
- Latency and egress cost

Pitfall:

- Cross-region replication without clear conflict strategy creates hard-to-debug data correctness issues.
-

17) Platform Engineering for Kafka

Treat Kafka as an internal platform product.

Platform capabilities:

- Self-service topic provisioning with policy guardrails
- Quota management by tenant/team
- Schema governance workflows
- Standardized retry/DLQ conventions
- Golden-path client templates/libraries
- Automated compliance and audit export

SRE/Platform KPIs:

- Cluster availability
- End-to-end produce/consume latency
- Rebalance frequency

- Consumer lag SLO attainment
 - Change failure rate for platform operations
-

18) Observability and SLOs

Metrics by layer:

- Broker: request latency, network throughput, disk utilization, under-replicated partitions, ISR count changes
- Producer: error rate, retries, record send rate, batch sizes
- Consumer: lag, poll latency, rebalance count/duration, processing latency

Golden signals:

- Traffic
- Errors
- Latency
- Saturation

SLO examples:

- P99 produce latency < X ms for critical topics
 - Consumer lag < Y records for Z minutes on critical groups
 - Under-replicated partitions sustained = 0 for business-critical clusters
-

19) Cost Engineering

Big cost drivers:

- Retention duration
- Replication factor
- Partition count
- Cross-AZ/region network
- Compression efficiency

Cost optimization levers:

- Right-size retention by topic tier
- Use compression wisely
- Archive cold data if required by policy
- Reduce over-partitioning
- Tiered storage where available and suitable

Pitfall:

- Cutting replication or durability to save cost can create hidden risk far more expensive than infra savings.
-

20) Common Design Patterns

Outbox Pattern

- Write business row + outbox event in one DB transaction.
- Relay publishes outbox to Kafka.
- Consumers remain idempotent.

CDC Pipeline

- Capture DB changes into Kafka.
- Build downstream projections/search/cache.

Event Sourcing + CQRS

- Event log as source of truth.
- Read models built asynchronously.

Retry Topology

- Main topic -> retry-1 -> retry-2 -> DLQ.
 - Include attempt metadata and reason codes.
-

21) Anti-Patterns (High Interview Value)

- Over-partitioning "for future" without benchmark.
 - No message key for entities requiring ordering.
 - Non-idempotent consumers with at-least-once semantics.
 - Retry loops without caps/jitter or DLQ.
 - Schema changes without compatibility enforcement.
 - Treating lag as only dashboard metric, not SLO.
 - Single shared principal for all apps (poor auditability/security).
 - Ignoring controller quorum health in KRaft clusters.
-

22) Interview Question Bank (Novice to Architect)

Beginner

1. What is a partition and why does it matter?
2. Why is ordering only guaranteed within a partition?
3. Difference between topic retention and compaction?

Intermediate

1. How do you achieve at-least-once and what duplicates appear?
2. How does consumer group rebalance impact latency?
3. Why use schema registry in microservice ecosystems?

Senior

1. Design a durable payment-events pipeline with low loss tolerance.
2. How do you choose partition count for 2-year growth?
3. What is your strategy for lag spikes and rebalance storms?
4. How do you secure multi-tenant Kafka platform access?

Architect

1. Design active-passive multi-region Kafka for RPO < 1 min and defined RTO.
 2. KRaft controller quorum sizing and placement strategy?
 3. Platform governance model for 100+ teams using Kafka?
 4. How do you evolve event contracts without deployment lockstep?
-

23) 90-Day Learning Roadmap (Novice -> Architect)

Days 1 to 15: Fundamentals

- Install local cluster and produce/consume basics.
- Practice partitioning, keys, and offset replay.

Days 16 to 35: Reliability

- Implement idempotent producer + manual commit consumer.
- Build retries + DLQ + observability dashboards.

Days 36 to 55: Data Contracts and Integration

- Add schema registry and compatibility checks.
- Build one Connect source and one sink pipeline.

Days 56 to 75: Performance and Operations

- Benchmark throughput/latency with config experiments.
- Tune batching/compression and consumer parallelism.

Days 76 to 90: Architecture

- Design DR/multi-cluster strategy.
 - Define platform standards (topic policy, quotas, security, onboarding).
 - Present trade-off memo with RPO/RTO/cost/risk.
-

24) Practical Checklists

Producer Checklist

- `acks=all`
- idempotence enabled
- retries configured
- key strategy reviewed for ordering/hotspots

Consumer Checklist

- manual commit after success
- bounded concurrency
- timeout + retry strategy
- idempotency key handling

Topic Checklist

- partitions sized to throughput and expected parallelism
- retention/compaction policy justified
- replication/min ISR meet durability target

Security Checklist

- authN + authZ enforced
- TLS enabled
- auditability enabled
- secrets rotation process defined

Platform Checklist

- self-service workflows with guardrails
 - quota/isolation model
 - observability with SLOs
 - DR runbook and game-day drills
-

25) Final Architect Summary

A 7+ YOE platform engineer or system architect is expected to:

- Explain Kafka internals (data plane + KRaft control plane) clearly.
- Convert business requirements into durability/latency/cost settings.
- Design for failures, not just throughput.
- Build governance around schemas, security, and multi-team platform usage.
- Operate Kafka with measurable SLOs and continuous reliability improvements.

If you can justify your decisions across correctness, scalability, operability, and cost, you are operating at architect level.

3. Why is lag not always bad?

- Temporary lag during spikes can be acceptable if SLO and recovery window are met.

4. What causes frequent rebalances?

- unstable consumers, long processing, heartbeat/session misconfiguration.

5. How to reduce duplicate processing impact?

- idempotent handlers and deterministic side effects.

13) Interview Story Prompts

- Scaled Kafka consumers for peak traffic without SLA breach.
- Reduced lag and rebalance storms with config/code changes.
- Built replay-safe event pipeline with schema governance.
- Handled incident caused by downstream DB slowness.

14) Practical Kafka Snippets and Explanations

14.1 Topic Creation with Operational Defaults

```
kafka-topics.sh --create \  
  --topic orders.events.v1 \  
  --partitions 12 \  
  --replication-factor 3 \  
  --config min.insync.replicas=2 \  
  --config retention.ms=604800000
```

Interview explanation:

- min.insync.replicas with acks=all raises durability guarantees.

14.2 Producer Config (Go sarama-style example)

```

cfg := sarama.NewConfig()
cfg.Producer.RequiredAcks = sarama.WaitForAll
cfg.Producer.Idempotent = true
cfg.Producer.Retry.Max = 5
cfg.Producer.Return.Successes = true

```

Interview explanation:

- Idempotent producer reduces duplicate risk from retries.

14.3 Consumer Pattern with Manual Commit Boundary

```

for msg := range messages {
    if err := handleMessage(msg); err != nil {
        sendToDLQ(msg, err)
        continue
    }
    markCommitted(msg)
}

```

Interview explanation:

- Commit after successful processing to preserve at-least-once semantics.

14.4 Lag Inspection Command

```

kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
    --group orders-consumer \
    --describe

```

Interview explanation:

- Lag spikes can come from slow downstream dependencies, not only Kafka itself.

15) L4 Kafka Architecture Drill

15.1 Durability and Throughput Trade-offs

- `acks=all + min.insync.replicas>=2` improves durability.
- Higher durability increases producer latency.
- Partition count drives parallelism and operational complexity.

15.2 Ordering and Keys

- Kafka ordering is guaranteed only within a partition.
- Key design determines partition mapping and therefore ordering semantics.
- Bad key cardinality creates hot partitions.

15.3 Consumer Group Rebalance Impact

- Rebalances can pause consumption and spike lag.
- Use cooperative rebalancing and right-size `max.poll.interval.ms`.
- Slow message handlers can look like broker problems.

15.4 Exactly-Once Semantics Boundaries

- EOS in Kafka is producer+transaction aware within Kafka ecosystem.
- External side effects (DB, email, HTTP call) still need idempotency.

Pattern:

```
consume event -> idempotency check -> side effect -> commit offset
```

15.5 Schema Governance

- Use schema registry with compatibility mode.
- Prefer additive evolution for backward compatibility.
- Reject unversioned payload contracts in production.

15.6 Senior Grilling Questions

1. Why is lag increasing if broker CPU is healthy?
 2. How do you replay safely without duplicating side effects?
 3. When do you choose Kafka vs RabbitMQ for a workflow?
 4. How do you reason about retention, compaction, and legal delete constraints?
-

16) L5 Kafka Platform Engineering

16.1 Throughput, Ordering, and Cost Triangle

Interview framing:

- More partitions increase parallelism but can increase ops complexity.
- Strong ordering requires key discipline and constrained partition strategy.
- Producer durability settings directly trade latency for safety.

16.2 Consumer Platform Standards

- Commit boundaries tied to idempotent side-effect completion.
- Retry policy with DLQ and poison-message handling.
- Rebalance-safe consumers with bounded processing time per poll.

16.3 Topic Governance

- Naming, ownership, retention classes, compaction policy.
- Schema compatibility gates in CI/CD.
- Access controls by producer/consumer principal boundaries.

17) Kafka Failure Mode Drills

17.1 Lag Explosion

Triage:

- rebalance churn,
- processing slowdown,
- partition hotspot,
- downstream dependency latency.

17.2 Duplicate Side Effects in Consumers

Triage:

- commit-before-side-effect bug,
- missing idempotency key,
- retry and timeout misalignment.

17.3 Ordering Violation Incident

Triage:

- key selection drift,
- producer batching/retry behavior,
- cross-topic ordering assumptions.

18) L5 Grilling Questions (Kafka)

1. How do you guarantee business correctness on at-least-once delivery?
2. Why does exactly-once in Kafka not mean exactly-once in your DB?
3. How do you model retention and compaction by business use case?
4. What changes first when traffic doubles: partitions, consumer instances, or code paths?
5. How do you canary consumer changes without risking group instability?

19) 60-Minute Kafka Revision Sprint

0-20 min:

- producer/consumer semantics and commit boundaries.

20-40 min:

- partitioning, ordering, lag, and schema governance.

40-60 min:

- one incident narrative: lag spike or duplicate processing with clear mitigation.
-